Database Systems(CSL3050)

Lab Assignment 3 - Report

Soumen Kumar
Roll No-B22ES006

1 Introduction

This report contains SQL queries for two database schemas: the University Database and the Movies Database. Each section includes a description of the queries executed, the SQL code used, and screenshots of the code and the output.

2 University Database

2.1 Schema Description

The University Database consists of several tables, including 'student', 'instructor', 'course', 'department', etc. The following queries were executed on this database:

2.2 Query Descriptions

2.2.1 Total Grade Points for a Student

Description: This query calculates the total grade points earned by the student with ID '12345' across all courses taken. The result is obtained by multiplying the grade points by the course credits and summing them up.

```
mysql> SELECT SUM(course.credit * grade_points.points) AS total_grade_points
    -> FROM takes
    -> JOIN course ON takes.course_id = course.course_id
    -> JOIN grade_points ON takes.grade = grade_points.grade
    -> WHERE takes.ID = 12345;
+--------------------+
| total_grade_points |
+--------------------+
|                 40 |
+--------------------+
1 row in set (0.00 sec)
```

Figure 1: Total Grade Points Earned by Student with ID 12345

2.2.2 GPA Calculation for a Student

Description: The GPA (Grade Point Average) for a student is calculated by dividing the total grade points by the total credits. This query returns the GPA for the student with a specific ID.

```
mysql> SELECT SUM(course.credit * grade_points.points) / SUM(course.credit) AS GPA
    -> FROM takes
    -> JOIN course ON takes.course_id = course.course_id
    -> JOIN grade_points ON takes.grade = grade_points.grade
    -> WHERE takes.ID = 12345;
+---------+
| GPA     |
+---------+
| 10.0000 |
+---------+
1 row in set (0.00 sec)
```

Figure 2: GPA for Student with ID 12345

2.2.3 GPA of Each Student in the CSE Department

Description: This query calculates the GPA for each student in the CSE department based on the courses they have taken. It returns the student ID and their GPA.

```
mysql> SELECT student.ID, SUM(course.credit * grade_points.points) / SUM(course.credit) AS GPA
    -> FROM student
    -> JOIN takes ON student.ID = takes.ID
    -> JOIN course ON takes.course_id = course.course_id
    -> JOIN grade_points ON takes.grade = grade_points.grade
    -> WHERE student.dept_name = 'CSE'
    -> GROUP BY student.ID;
+------+---------+
| ID   | GPA     |
+------+---------+
| 2001 | 10.0000 |
+------+---------+
1 row in set (0.00 sec)
```

Figure 3: GPA of Each Student in the CSE Department

2.2.4 Department Topper

Description: This query finds the student with the highest GPA in each department. The result includes the student's ID, name, department, and their GPA.
Step 1: Find the maximum GPA for each department.
Step 2: Identify the student(s) with that maximum GPA in each department.

```
mysql> SELECT dept_name, MAX(GPA) AS Top_GPA
    -> FROM (
    ->     SELECT student.dept_name,
    ->            SUM(course.credit * grade_points.points) / SUM(course.credit) AS GPA
    ->     FROM student
    ->     JOIN takes ON student.ID = takes.ID
    ->     JOIN course ON takes.course_id = course.course_id
    ->     JOIN grade_points ON takes.grade = grade_points.grade
    ->     GROUP BY student.ID
    -> ) AS department_gpas
    -> GROUP BY dept_name;
+-------------+---------+
| dept_name   | Top_GPA |
+-------------+---------+
| ES          | 10.0000 |
| CSE         | 10.0000 |
| EE          |  8.0000 |
| Physics     |  9.0000 |
| Mathematics |  8.0000 |
| English     |  7.0000 |
| Art         |  6.0000 |
| Music       |  7.0000 |
| Business    |  9.0000 |
+-------------+---------+
9 rows in set (0.00 sec)

mysql> SELECT student.ID, student.name, student.dept_name, department_gpas.Top_GPA
    -> FROM student
    -> JOIN (
    ->     SELECT dept_name, MAX(GPA) AS Top_GPA
    ->     FROM (
    ->         SELECT student.dept_name,
    ->                SUM(course.credit * grade_points.points) / SUM(course.credit) AS GPA
    ->         FROM student
    ->         JOIN takes ON student.ID = takes.ID
    ->         JOIN course ON takes.course_id = course.course_id
    ->         JOIN grade_points ON takes.grade = grade_points.grade
    ->         GROUP BY student.ID
    ->     ) AS department_gpas
    ->     GROUP BY dept_name
    -> ) AS department_gpas ON student.dept_name = department_gpas.dept_name
    -> JOIN (
    ->     SELECT student.ID, SUM(course.credit * grade_points.points) / SUM(course.credit) AS GPA
    ->     FROM student
    ->     JOIN takes ON student.ID = takes.ID
    ->     JOIN course ON takes.course_id = course.course_id
    ->     JOIN grade_points ON takes.grade = grade_points.grade
    ->     GROUP BY student.ID
    -> ) AS student_gpas ON student.ID = student_gpas.ID
    -> AND student_gpas.GPA = department_gpas.Top_GPA;
+-------+---------+-------------+---------+
| ID    | name    | dept_name   | Top_GPA |
+-------+---------+-------------+---------+
| 12345 | Soumen  | ES          | 10.0000 |
|  2001 | David   | CSE         | 10.0000 |
|  2002 | Eva     | EE          |  8.0000 |
|  2003 | Frank   | Physics     |  9.0000 |
|  2004 | Grace   | Mathematics |  8.0000 |
|  2005 | Henry   | English     |  7.0000 |
|  2008 | Karen   | Art         |  6.0000 |
|  2009 | Michael | Music       |  7.0000 |
|  2010 | Olivia  | Business    |  9.0000 |
+-------+---------+-------------+---------+
```

Figure 4: Topper of Each Department

2.2.5 Departments with Higher Budgets than Philosophy

Description: This query lists all department names with budgets higher than the Philosophy department's budget. The result is ordered alphabetically.

```
mysql> SELECT dept_name
    -> FROM department
    -> WHERE budget > (
    ->     SELECT budget
    ->     FROM department
    ->     WHERE dept_name = 'Philosophy'
    -> )
    -> ORDER BY dept_name;
+-------------+
| dept_name   |
+-------------+
| Biology     |
| Business    |
| CSE         |
| EE          |
| English     |
| History     |
| Mathematics |
| Music       |
| Physics     |
+-------------+
9 rows in set (0.00 sec)
```

Figure 5: Departments with Higher Budgets than Philosophy

2.2.6 Students Taking at Least Three Distinct Courses

Description: This query identifies students who have taken at least three distinct courses. It returns the IDs of such students.

```
mysql> SELECT ID
    -> FROM takes
    -> GROUP BY ID
    -> HAVING COUNT(DISTINCT course_id) >= 3;
Empty set (0.00 sec)
```

Figure 6: Students Taking at Least Three Distinct Courses

2.2.7 Instructors Teaching Every Course in Their Department

Description: This query finds instructors who teach every course offered by their department. The result includes the instructor's name and ID.

```
mysql> SELECT i.ID, i.name
    -> FROM instructor i
    -> JOIN (
    ->     SELECT dept_name, COUNT(*) AS total_courses
    ->     FROM course
    ->     GROUP BY dept_name
    -> ) AS dc ON i.dept_name = dc.dept_name
    -> JOIN (
    ->     SELECT t.ID, c.dept_name, COUNT(DISTINCT c.course_id) AS taught_courses
    ->     FROM teaches t
    ->     JOIN course c ON t.course_id = c.course_id
    ->     GROUP BY t.ID, c.dept_name
    -> ) AS tc ON i.ID = tc.ID AND i.dept_name = tc.dept_name
    -> WHERE tc.taught_courses = dc.total_courses;
+-----+----------+
| ID  | name     |
+-----+----------+
| 101 | Alice    |
| 102 | Bob      |
| 104 | David    |
| 105 | Emily    |
| 106 | Frank    |
| 107 | Grace    |
| 108 | Henry    |
| 109 | Isabella |
| 110 | Jack     |
| 111 | Karen    |
+-----+----------+
10 rows in set (0.00 sec)
```

Figure 7: Instructors Teaching Every Course in Their Department

2.2.8 Instructors with More Than Three A+ Grades in Fall 2018

Description: This query lists instructors who have awarded more than three A+ grades in a course during the Fall 2018 semester. The result includes the instructor's name, ID, and course details.

```
mysql> SELECT i.ID, i.name, c.course_id, c.title
    -> FROM instructor i
    -> JOIN teaches t ON i.ID = t.ID
    -> JOIN course c ON t.course_id = c.course_id
    -> JOIN takes tk ON t.course_id = tk.course_id AND t.sec_id = tk.sec_id AND t.semester = tk.semester AND t.year = tk.year
    -> WHERE tk.grade = 'A+' AND t.semester = 'Fall' AND t.year = 2018
    -> GROUP BY i.ID, i.name, c.course_id, c.title
    -> HAVING COUNT(tk.ID) > 3;
Empty set (0.01 sec)
```

Figure 8: Instructors with More Than Three A+ Grades in Fall 2018

2.2.9 Instructors Who Have Never Given an A Grade

Description: This query identifies instructors who have never awarded an A grade in any of their courses. It returns the names and IDs of such instructors.

```
mysql> SELECT DISTINCT i.ID, i.name
    -> FROM instructor i
    -> JOIN teaches t ON i.ID = t.ID
    -> WHERE NOT EXISTS (
    ->     SELECT 1
    ->     FROM takes tk
    ->     WHERE t.course_id = tk.course_id AND t.sec_id = tk.sec_id
    ->     AND t.semester = tk.semester AND t.year = tk.year
    ->     AND (tk.grade = 'A+' OR tk.grade = 'A')
    -> );
+-----+----------+
| ID  | name     |
+-----+----------+
| 102 | Bob      |
| 105 | Emily    |
| 106 | Frank    |
| 107 | Grace    |
| 108 | Henry    |
| 109 | Isabella |
| 110 | Jack     |
+-----+----------+
7 rows in set (0.01 sec)
```

Figure 9: Instructors Who Have Never Given an A Grade

2.2.10 View for Student Grades (studentgrades)

Description: This view provides the GPA of each student, ensuring proper handling of null values in the grade attribute. It simplifies retrieving GPA information for all students.

```
mysql> CREATE VIEW student_grades AS
    -> SELECT s.ID,
    ->        COALESCE(SUM(c.credit * gp.points) / SUM(c.credit), 0) AS GPA
    -> FROM student s
    -> LEFT JOIN takes t ON s.ID = t.ID
    -> LEFT JOIN course c ON t.course_id = c.course_id
    -> LEFT JOIN grade_points gp ON t.grade = gp.grade
    -> GROUP BY s.ID;
ERROR 1050 (42S01): Table 'student_grades' already exists
mysql> SELECT * FROM student_grades;
+-------+---------+
| ID    | GPA     |
+-------+---------+
|  2001 | 10.0000 |
|  2002 |  8.0000 |
|  2003 |  9.0000 |
|  2004 |  8.0000 |
|  2005 |  7.0000 |
|  2006 |  0.0000 |
|  2007 |  0.0000 |
|  2008 |  6.0000 |
|  2009 |  7.0000 |
|  2010 |  9.0000 |
| 12345 | 10.0000 |
+-------+---------+
11 rows in set (0.00 sec)
```

Figure 10: View for Student Grades (studentgrades)

3 Movies Database

3.1 Schema Description

The Movies Database consists of several tables, including ‘movie‘, ‘actor‘, ‘director‘, ‘rating‘, etc. The following queries were executed on this database:

3.2 Query Descriptions

3.2.1 Movies Released Before 2000

Description: This query finds the total number of movies released before the year 2000.

```
mysql> SELECT COUNT(*) AS total_movies_before_2000
    -> FROM movie
    -> WHERE release_year < 2000;
+--------------------------+
| total_movies_before_2000 |
+--------------------------+
|                        2 |
+--------------------------+
1 row in set (0.00 sec)
```

Figure 11: Total Number of Movies Released Before 2000

3.2.2 Average Rating of Action Movies

Description: This query calculates the average rating for all movies in the ’Action’ genre.

```
mysql> SELECT AVG(r.rating_score) AS avg_action_rating
    -> FROM movie m
    -> JOIN rating r ON m.movie_id = r.movie_id
    -> WHERE m.genre = 'Action';
+-------------------+
| avg_action_rating |
+-------------------+
|              NULL |
+-------------------+
1 row in set (0.00 sec)
```

Figure 12: Average Rating of Action Movies

3.2.3 Movies with High Average Rating

Description: This query identifies movies with an average rating higher than 4.5. It includes the movie ID and title.

```
mysql> SELECT m.movie_id, m.title
    -> FROM movie m
    -> JOIN rating r ON m.movie_id = r.movie_id
    -> GROUP BY m.movie_id, m.title
    -> HAVING AVG(r.rating_score) > 4.5;
+----------+------------+
| movie_id | title      |
+----------+------------+
|        1 | Inception  |
|        2 | The Matrix |
|        3 | Titanic    |
+----------+------------+
3 rows in set (0.00 sec)
```

Figure 13: Movies with Average Rating Higher Than 4.5

3.2.4 Directors Born Before 1970

Description: This query lists the names of all directors who were born before the year 1970.

```
mysql> SELECT name
    -> FROM director
    -> WHERE birthdate < '1970-01-01';
+---------------+
| name          |
+---------------+
| James Cameron |
+---------------+
1 row in set (0.00 sec)
```

Figure 14: Directors Born Before 1970

3.2.5 Most Recent Drama Movie

Description: This query finds the most recent movie in the 'Drama' genre, returning the title and release year.

```
mysql> SELECT title, release_year
    -> FROM movie
    -> WHERE genre = 'Drama'
    -> ORDER BY release_year DESC
    -> LIMIT 1;
Empty set (0.00 sec)
```

Figure 15: Most Recent Drama Movie

3.2.6 Top 3 Highest-Rated Movies

Description: This query lists the top 3 highest-rated movies along with their titles, genres, and average ratings.

```
mysql> SELECT m.title, m.genre, AVG(r.rating_score) AS avg_rating
    -> FROM movie m
    -> JOIN rating r ON m.movie_id = r.movie_id
    -> GROUP BY m.movie_id, m.title, m.genre
    -> ORDER BY avg_rating DESC
    -> LIMIT 3;
+------------+---------+------------+
| title      | genre   | avg_rating |
+------------+---------+------------+
| Titanic    | Romance |    9.50000 |
| Inception  | Sci-Fi  |    9.00000 |
| The Matrix | Sci-Fi  |    8.50000 |
+------------+---------+------------+
3 rows in set (0.00 sec)
```

Figure 16: Top 3 Highest-Rated Movies

3.2.7 Actors in More Than 5 Movies

Description: This query identifies actors who have appeared in more than 5 movies, listing their names.

```
mysql> SELECT a.name
    -> FROM actor a
    -> JOIN cast c ON a.actor_id = c.actor_id
    -> GROUP BY a.actor_id, a.name
    -> HAVING COUNT(c.movie_id) > 5;
Empty set (0.00 sec)
```

Figure 17: Actors in More Than 5 Movies

3.2.8 Movies Directed by Directors of Multiple Comedy Movies

Description: This query finds all movies directed by directors who have directed at least 2 movies in the 'Comedy' genre.

```
mysql> SELECT m.title
    -> FROM movie m
    -> JOIN directs d ON m.movie_id = d.movie_id
    -> WHERE d.director_id IN (
    ->     SELECT director_id
    ->     FROM movie m
    ->     JOIN directs d ON m.movie_id = d.movie_id
    ->     WHERE m.genre = 'Comedy'
    ->     GROUP BY d.director_id
    ->     HAVING COUNT(m.movie_id) >= 2
    -> );
Empty set (0.00 sec)
```

Figure 18: Movies Directed by Directors of Multiple Comedy Movies

3.2.9 Users Who Have Rated At Least 10 Movies

Description: This query lists the names and IDs of users who have rated at least 10 distinct movies.

```
mysql> SELECT u.user_id, u.name
    -> FROM user u
    -> JOIN rating r ON u.user_id = r.user_id
    -> GROUP BY u.user_id, u.name
    -> HAVING COUNT(DISTINCT r.movie_id) >= 10;
Empty set (0.00 sec)
```

Figure 19: Users Who Have Rated At Least 10 Movies

3.2.10 View for Movie Ratings (movieratings)

Description: This view provides the average rating of each movie, ensuring proper handling of null values in the rating attribute. It simplifies retrieving average ratings for all movies.

```
mysql> CREATE VIEW movie_ratings AS
    -> SELECT m.movie_id, AVG(r.rating_score) AS avg_rating
    -> FROM movie m
    -> LEFT JOIN rating r ON m.movie_id = r.movie_id
    -> GROUP BY m.movie_id;
Query OK, 0 rows affected (0.02 sec)

mysql> select * from movie_ratings;
+----------+------------+
| movie_id | avg_rating |
+----------+------------+
|        1 |    9.00000 |
|        2 |    8.50000 |
|        3 |    9.50000 |
+----------+------------+
3 rows in set (0.00 sec)
```

Figure 20: View for Movie Ratings (movieratings)

4 Conclusion

This report documented the SQL queries executed on both the University and Movies databases. Screenshots of the code and outputs were included to demonstrate the results of the queries.